S-Lang

User Guide and Language Reference

Version 1.0

A practical introduction and normative reference for the S-Lang programming language

S-Lang Project
XeLaTeX or LuaLaTeX

How to Use This Book

This book combines a practical course with a language reference. The first chapters introduce the language step by step. Later chapters describe the accepted syntax, standard-library surface, command-line tools, and important limitations.

If you are new to S-Lang, read the chapters in order and type the examples into `.sl` files. Run `slang check` before running a program. Modify example values and observe the result. If you already know the language, use the table of contents and the reference chapters.

The compiler is the authority for accepted syntax. A construct accepted by the parser is not automatically supported by every execution backend. The interpreter and Native JIT may have different capability boundaries; use the backend capability command when that distinction matters.

Learning Path

1. Foundations: source files, comments, values, variables, expressions, and conditions.
2. Programming: functions, records, entities, collections, and pipelines.
3. Applications: modules, files, CSV, JSON, and standard-library functions.
4. Advanced topics: declarative constructs, native execution, diagnostics, and backend limitations.

Contents

1 What Is S-Lang? 1
- 1.1 Design principles . . . 1
- 1.2 Version and compatibility . . . 1

2 Installation and First Steps 2
- 2.1 Check the installation . . . 2
- 2.2 Create and run a source file . . . 2
- 2.3 Create a project . . . 2

3 Source Files, Comments, and Basic Syntax 3
- 3.1 Source files and names . . . 3
- 3.2 Comments . . . 3
- 3.3 Blocks . . . 3

4 Values, Variables, and Expressions 4
- 4.1 Literals . . . 4
- 4.2 Field access and function calls . . . 4
- 4.3 Operators and precedence . . . 4

5 Bindings and Mutable State 6

6 Conditions 7
- 6.1 Conditional expressions . . . 7
- 6.2 Block conditions . . . 7

7 Functions 8
- 7.1 Declaring a function . . . 8
- 7.2 Generic functions . . . 8

8 Types, Records, Entities, and Properties 9
- 8.1 Common types . . . 9
- 8.2 Records . . . 9
- 8.3 Entities and properties . . . 9

9 Collections 11
- 9.1 Filtering with `where` . . . 11
- 9.2 Transforming with `map` . . . 11
- 9.3 Sorting . . . 11
- 9.4 Aggregates . . . 11
- 9.5 Reduction . . . 12

10 Pipelines 13
- 10.1 Explicit and compact forms . . . 13
- 10.2 Conditional stages and `tee` . . . 13

11 Sequences and Declarative Constructs **14**
11.1 Sequences . . . 14
11.2 Constraint-based solutions . . . 14
11.3 Data tables and graphs . . . 14
11.4 Physical models . . . 15

12 Modules and Program Organization **16**
12.1 Modules and imports . . . 16
12.2 Exports and re-exports . . . 16

13 Input, Output, and the Standard Library **17**
13.1 Reading CSV and JSON . . . 17
13.2 Public standard-library catalog . . . 17
13.2.1 aggregate — Grouping and aggregation . . . 17
13.2.2 analytics — Statistical and analytical operations . . . 18
13.2.3 collections — Collection processing . . . 18
13.2.4 csv — CSV input and output . . . 22
13.2.5 data — Record and field access . . . 22
13.2.6 datetime — Date and time operations . . . 23
13.2.7 files — File and directory operations . . . 23
13.2.8 json — JSON parsing and serialization . . . 24
13.2.9 math — Mathematical functions . . . 24
13.2.10 path — File path handling . . . 26
13.2.11 process — Process and environment management . . . 26
13.2.12 relational — Relational data operations . . . 27
13.2.13 text — Text processing . . . 28

14 Command-Line Reference **30**

15 Diagnostics and Validation Workflow **31**

16 Forms Not Included in S-Lang 1.0 **32**

17 Complete Example **33**

18 Practice Exercises **34**
18.1 Beginner . . . 34
18.2 Intermediate . . . 34
18.3 Applied . . . 34

19 Language Reference and Change Control **35**

Chapter 1

What Is S-Lang?

S-Lang is a high-level, general-purpose programming language. Its syntax emphasizes describing tasks, data, and transformations rather than spelling out low-level implementation details. The design goal is readable, deterministic source code that can be understood by ordinary users as well as experienced developers.

The language includes values and expressions, explicit variable declarations, mutable state, functions, records, entities, collection operations, pipelines, modules, a standard library, and domain-oriented constructs for selected computational tasks.

1.1 Design principles

- Prefer clear, explicit constructs over compact but ambiguous syntax.
- Keep grammar deterministic and documented; do not assume that syntax from Python, JavaScript, Rust, or SQL is accepted.
- Use `let` for ordinary bindings and `state`/`set` for mutable state.
- Keep pipeline stages on one physical source line.
- Treat parser, semantic validation, runtime behavior, tests, and documentation as separate but coordinated contracts.
- Preserve readability when adding language features. A shorter program is not automatically a clearer program.

1.2 Version and compatibility

This book describes the S-Lang 1.0 language line. The public version is `1.0`. Package metadata may use the same version string; the Windows executable is distributed as `slang1.0.exe`. The exact accepted syntax is defined by the compiler shipped with the release and by the conformance tests included in the source package.

Chapter 2

Installation and First Steps

2.1 Check the installation

Open a terminal and run:

```
slang version
```

The command should report `1.0`. On Windows, the installed executable is named `slang1.0.exe`; use that filename when invoking it directly. If the installer adds its directory to the user PATH, the executable can be started from a new terminal.

2.2 Create and run a source file

Create a text file named `main.sl`:

```
let name = "S-Lang"
let release = 1.0
show "Hello from", name
show "Version:", release
```

Check and run it:

```
slang check main.sl
slang run main.sl
```

The command name shown above is the standard command on Unix-like systems and for installations that provide a command alias. On Windows, use `slang1.0.exe` if no `slang` alias is provided.

2.3 Create a project

```
slang init my-app --name my-app --version 0.1.0
cd my-app
slang check src/main.sl
slang run src/main.sl
```

The version passed to `slang init` is the version of the project being created, not the installed compiler version.

Chapter 3

Source Files, Comments, and Basic Syntax

3.1 Source files and names

S-Lang source files use the `.sl` extension. Names are case-sensitive. Use spaces consistently for indentation and do not mix tabs and spaces in the same block.

3.2 Comments

The `\#` character begins a comment outside a quoted string. A comment continues to the end of the physical line. It may appear on a line by itself or after an instruction. A hash character inside single- or double-quoted text is part of the string.

```
# A full-line comment
let label = "S-Lang # language" # A trailing comment
let note = 'The # character is text here'
show label, note
```

3.3 Blocks

A block header ends with a colon. Its body is indented:

```
if true:
    show "The condition is true"
else:
    show "The condition is false"
```

The `else:` line must align with its matching `if:`. Empty branches are not valid.

Chapter 4

Values, Variables, and Expressions

4.1 Literals

```
let whole = 12
let decimal = 2.5
let scientific = 1.25e3
let enabled = true
let disabled = false
let title = "S-Lang"
let alternate = 'text'
let values = [1, 2, 3]
let person = {name: "Ada", age: 36}
let empty_values = []
let empty_record = {}
```

The language supports integer, decimal, and scientific-notation numbers; Boolean values; strings; lists; and record literals. Record keys may be identifiers or quoted strings.

4.2 Field access and function calls

```
let person = {name: "Ada", age: 36}
let person_name = person.name
let normalized = upper(person.name)
let root = sqrt(81)
show person_name, normalized, root
```

A function call has the form `name(argument1, argument2)`. Qualified names may use a dot. Record fields are accessed with dot notation. List indexing such as `values[0]` is not part of the S-Lang 1.0 grammar.

4.3 Operators and precedence

From highest to lowest, the expression precedence is:

Level	Operators/forms
1	Parentheses, literals, names, calls, dot field access
2	Exponentiation: `\^`
3	Unary `+`, `-`, `not`
4	Multiplication, division, remainder: `*`, `/`, `\%`
5	Addition and subtraction: `+`, `-`
6	Comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=`
7	Logical `and`
8	Logical `or`
9	Conditional expression: `if ... then ... else ...`

Exponentiation is right-associative: `2 ^ 3 ^ 2` means `2 ^ (3 ^ 2)`. Unary minus binds outside exponentiation. Do not write chained comparisons such as `a < b < c`; write `a < b and b < c`.

```
let a = 12
let b = 5
let total = a + b
let quotient = a / b
let remainder = a % b
let power = 2 ^ 3
let allowed = a >= 10 and b != 0
let label = if allowed then "valid" else "invalid"
show total, quotient, remainder, power, label
```

Chapter 5

Bindings and Mutable State

Use `let` for an ordinary binding:

```
let title = "Report"
let count = 3
show title, count
```

Use `state` to declare a value that will be updated, and `set` to update it:

```
state counter = 0
set counter = counter + 1
show counter
```

The `set` statement updates an existing state binding; it does not declare a new variable. Do not use bare assignment as a universal declaration form.

Chapter 6

Conditions

6.1 Conditional expressions

A conditional expression produces one value:

```
let score = 82
let grade = if score >= 90 then "A" else if score >= 75 then "B" else "C"
show grade
```

6.2 Block conditions

Use an indented block when a branch contains one or more statements:

```
let load = 68
if load >= 90:
    show "Critical"
else:
    if load >= 70:
        show "High"
    else:
        show "Normal"
```

Conditions must evaluate to Boolean values. Keep indentation consistent and align each `else:` with its matching `if:`.

Chapter 7

Functions

7.1 Declaring a function

```
function square(value: Number) -> Number:
    return value * value

let result = square(9)
show result
```

Parameters are comma-separated and use the form `name: Type`. A parameter type may be omitted where inference is sufficient. A return type after `->` may also be omitted. Public functions are easier to understand when their parameter and result types are explicit. The function body must return a value on every path when a result is declared or required by its use.

7.2 Generic functions

```
function identity<T>(value: T) -> T:
    return value

let number = identity(7)
let text_value = identity("S-Lang")
show number, text_value
```

Type parameters are declared in angle brackets. Use distinct type-parameter names within a declaration.

Chapter 8

Types, Records, Entities, and Properties

8.1 Common types

Common type names include `Number`, `Boolean`, `Text`, `List`, `Record`, and `Entity`. Parameterized forms include `List<Number>` and function types in supported type contexts. The semantic analyzer checks type compatibility and reports invalid field access or incompatible expressions.

8.2 Records

A record groups named fields:

```
record Person:
    name: Text
    age: Number

let ada = Person("Ada", 36)
show ada.name, ada.age
```

Record fields must have unique names. A record declaration must contain at least one field. Generic records are supported:

```
record Box<T>:
    value: T

let box = Box(7)
show box.value
```

8.3 Entities and properties

Entities can describe domain objects with measured fields:

```
entity Circle:
    radius: m

let disk = Circle(10 m)
show disk.radius
```

A property is attached to a declared entity. Inside the property body, `self` refers to the current instance:

```
entity Circle:
    radius: m

property Circle.diameter -> m:
    return self.radius * 2

let disk = Circle(10 m)
show disk.diameter
```

Chapter 9

Collections

Collection operations provide a declarative way to filter, transform, sort, and summarize lists.

9.1 Filtering with `where`

```
let values = [1, 2, 3, 4, 5]
let selected = values where item >= 3
show selected
```

The predicate must return a Boolean value. In an element context, `item` or `row` refers to the current element. For record collections, access a field with `row.field` or `item.field`.

9.2 Transforming with `map`

```
let values = [1, 2, 3]
let doubled = values map item * 2
show doubled
```

9.3 Sorting

```
let values = [3, 1, 2]
let ascending = values sort by item asc
let descending = values sort by item desc
show ascending, descending
```

The direction may be omitted; ascending order is the default.

9.4 Aggregates

```
let values = [1, 2, 3, 4]
let total = values sum
let amount = values count
let largest = values max
let smallest = values min
let mean = values average
```

```
show total, amount, largest, smallest, mean
```

The supported aggregate names are `sum`, `count`, `max`, `min`, `average`, `avg`, `any`, and `all`. The `count` operation does not need an element expression. Other aggregates use the current element by default; an explicit expression such as `values sum item` may be supplied.

9.5 Reduction

The `reduce` operation combines the elements into one result. `acc` is the accumulator; `item` or `row` is the current element:

```
let values = [1, 2, 3, 4]
let total = values reduce 0 with acc + item
let product = values reduce 1 with acc * item
show total, product
```

For record collections, use `row.field` or `item.field`. A bare field name can refer to a field of the current record in collection context, but it is not an alias for a scalar list item. Thus a numeric list reduction uses `item`, not `value`:

```
data measurements:
    first value 10
    second value 20
let total = measurements reduce 0 with acc + row.value
show total
```

The step expression must return a value compatible with the initial accumulator.

Chapter 10

Pipelines

A pipeline connects collection operations in a readable sequence. In S-Lang 1.0, the complete pipeline must be written on one physical source line.

```
let numbers = [1, 2, 3, 4, 5]
let result = numbers |> where item >= 3 |> map item * 2 |> sort by item desc |> sum
show result
```

10.1 Explicit and compact forms

Inside a pipeline, `item` and `row` refer to the current element. A leading comparison or arithmetic operator can be used as shorthand for the current element:

```
let numbers = [1, 2, 3, 4, 5]
let compact = numbers |> where >= 3 |> map * 2 |> sum
show compact
```

For record collections, a leading field such as `.score` is shorthand for the current record's field:

```
let records = [{score: 10}, {score: 20}, {score: 30}]
let high = records |> where .score >= 20 |> sort by .score desc
show high
```

Use the explicit `item.field` or `row.field` form when it improves clarity.

10.2 Conditional stages and `tee`

Conditional pipeline stages select a branch based on a condition. `tee` allows a pipeline value to be observed while the pipeline continues. Because these forms have additional context rules, follow the exact examples and constraints in the compiler's normative reference and tests. Do not split a pipeline across physical lines.

Chapter 11

Sequences and Declarative Constructs

11.1 Sequences

A sequence declaration describes a deterministic series:

```
sequence doubled from 1 for 5 steps using value * 2
sequence bounded from 1 for up to 10 steps until value >= 100 using value * 2
```

The first form specifies a fixed number of steps. The second form specifies an upper bound and a stopping condition.

11.2 Constraint-based solutions

The `satisfy` construct describes constraints over declared variables:

```
solution = satisfy:
    x in 1..10
    y in 1..10
    x + y == 12
show solution
```

The accepted structure is a domain construct, not a general-purpose block expression. Use the form documented by the compiler and its tests.

11.3 Data tables and graphs

Data tables provide named rows and fields:

```
data students:
    Alice age 22 score 91
    Bob age 17 score 88

let adults = students where age >= 18
let ranked = adults sort by score desc
show ranked
```

Graph declarations describe nodes and weighted connections:

```
graph routes:
    A -> B cost 2
```

```
    B <-> C cost 3
```

11.4 Physical models

Physical quantities can be written with units. A model may declare bodies and a simulation:

```
physics gravity
body sun:
    mass 1 kg
    position 0 m
    velocity 0 mps
body earth:
    mass 1 kg
    position 1 m
    velocity 1 mps
simulate earth around sun for 1 day step 1 minute
```

The exact units and domain constraints are checked by the semantic analyzer and runtime. Do not infer that every model is supported by every backend.

Chapter 12

Modules and Program Organization

12.1 Modules and imports

A module declaration names a source module. Imports may use a file path or a registered standard module:

```
module geometry
import "geometry.sl" as geo
import std.math
import "geometry.sl" {Circle, circumference}
```

The exact forms of imports are resolved by the module loader. Standard module names are registered in `slang/stdlib/modules.py`.

12.2 Exports and re-exports

A module can expose public names and re-export selected names from another module:

```
export Circle, circumference
export {Circle} from "geometry.sl"
```

Only exported names form the public module API. Private names are not implicitly re-exported.

Chapter 13

Input, Output, and the Standard Library

The standard library includes mathematics, text, collections, data access, aggregation, relational operations, analytics, JSON, CSV, paths, files, processes, and date/time utilities.

13.1 Reading CSV and JSON

```
let rows = read_csv("data/people.csv")
let people = rows where item.age >= 18
show people
```

```
let value = read_json("data/config.json")
show value
```

File operations depend on filesystem access. Process operations may start external programs and should be used deliberately. Check the function's effect and capability in the standard-library catalog before relying on it in a restricted runtime.

13.2 Public standard-library catalog

The following table is generated from the public catalog in `slang/stdlib/catalog.py`. It lists all public function names in the S-Lang 1.0 source tree, grouped by module. Function contracts, argument types, and effect constraints are defined by the implementation and semantic registry.

13.2.1 aggregate — Grouping and aggregation

Function	Description
`frequency`	Standard-library grouping and aggregation operation: frequency.
`group_average`	Standard-library grouping and aggregation operation: group average.
`group_by`	Group records by the selected field.
`group_count`	Count records in each field-value group.

Function	Description
group_max	Standard-library grouping and aggregation operation: group max.
group_min	Standard-library grouping and aggregation operation: group min.
group_sum	Sum a numeric field within each group.
mode	Standard-library grouping and aggregation operation: mode.

13.2.2 analytics — Statistical and analytical operations

Function	Description
bottom_by	Standard-library data analytics operation: bottom by.
group_top_by	Standard-library data analytics operation: group top by.
rank_by	Standard-library data analytics operation: rank by.
top_by	Standard-library data analytics operation: top by.
window	Split records into sliding windows.
window_average	Standard-library data analytics operation: window average.
window_sum	Standard-library data analytics operation: window sum.

13.2.3 collections — Collection processing

Function	Description
aggregate_groups	Standard-library collection processing operation: aggregate groups.
aggregate_values_by	Standard-library collection processing operation: aggregate values by.
aggregate_values_by_with_key	Standard-library collection processing operation: aggregate values by with key.
all_true	Standard-library collection processing operation: all true.
any_true	Standard-library collection processing operation: any true.
argmax_number	Standard-library collection processing operation: argmax number.
argmin_number	Standard-library collection processing operation: argmin number.
average_number	Return the arithmetic mean of a list of numbers.
chunk	Standard-library collection processing operation: chunk.
chunk_exact	Standard-library collection processing operation: chunk exact.
clip_numbers	Standard-library collection processing operation: clip numbers.

Function	Description
concat_values	Standard-library collection processing operation: concat values.
contains_all_values	Standard-library collection processing operation: contains all values.
contains_any_value	Standard-library collection processing operation: contains any value.
contains_value	Standard-library collection processing operation: contains value.
correlation_numbers	Standard-library collection processing operation: correlation numbers.
cosine_similarity_numbers	Standard-library collection processing operation: cosine similarity numbers.
count_by_values	Standard-library collection processing operation: count by values.
count_if	Standard-library collection processing operation: count if.
count_value	Standard-library collection processing operation: count value.
covariance_numbers	Standard-library collection processing operation: covariance numbers.
cumulative_average_numbers	Standard-library collection processing operation: cumulative average numbers.
cumulative_max_numbers	Standard-library collection processing operation: cumulative max numbers.
cumulative_min_numbers	Standard-library collection processing operation: cumulative min numbers.
cumulative_product_numbers	Standard-library collection processing operation: cumulative product numbers.
cumulative_sum_numbers	Standard-library collection processing operation: cumulative sum numbers.
difference_values	Standard-library collection processing operation: difference values.
differences_numbers	Standard-library collection processing operation: differences numbers.
distinct_by_values	Standard-library collection processing operation: distinct by values.
dot_product_numbers	Standard-library collection processing operation: dot product numbers.
drop_until	Standard-library collection processing operation: drop until.
drop_while	Standard-library collection processing operation: drop while.
enumerate_values	Standard-library collection processing operation: enumerate values.
euclidean_distance_numbers	Standard-library collection processing operation: euclidean distance numbers.
filter_values	Standard-library collection processing operation: filter values.
find_all	Return all items matching a condition.
find_all_indices	Standard-library collection processing operation: find all indices.

Function	Description
find_first	Return the first item matching a condition.
find_last	Return the last item matching a condition.
flatten_one	Standard-library collection processing operation: flatten one.
fold_values	Standard-library collection processing operation: fold values.
group_by_values	Standard-library collection processing operation: group by values.
group_values_by	Standard-library collection processing operation: group values by.
has_at_least_values	Standard-library collection processing operation: has at least values.
has_at_most_values	Standard-library collection processing operation: has at most values.
has_exactly_values	Standard-library collection processing operation: has exactly values.
index_by_values	Standard-library collection processing operation: index by values.
index_of_number	Standard-library collection processing operation: index of number.
intersect_values	Standard-library collection processing operation: intersect values.
is_empty_values	Standard-library collection processing operation: is empty values.
is_multiple_values	Standard-library collection processing operation: is multiple values.
is_not_empty_values	Standard-library collection processing operation: is not empty values.
is_single_value	Standard-library collection processing operation: is single value.
is_unique_values	Standard-library collection processing operation: is unique values.
l2_norm_numbers	Standard-library collection processing operation: l2 norm numbers.
last_index_of_number	Standard-library collection processing operation: last index of number.
manhattan_distance_numbers	Standard-library collection processing operation: manhattan distance numbers.
map_values	Standard-library collection processing operation: map values.
map_with_index	Standard-library collection processing operation: map with index.
max_by_values	Standard-library collection processing operation: max by values.
max_number	Standard-library collection processing operation: max number.
median_number	Return the median of a list of numbers.
min_by_values	Standard-library collection processing operation: min by values.
min_number	Standard-library collection processing operation: min number.

Function	Description
moving_average_numbers	Standard-library collection processing operation: moving average numbers.
none_true	Standard-library collection processing operation: none true.
normalize_numbers	Standard-library collection processing operation: normalize numbers.
pairwise	Standard-library collection processing operation: pairwise.
partition_by_values	Standard-library collection processing operation: partition by values.
partition_indices	Standard-library collection processing operation: partition indices.
partition_values	Standard-library collection processing operation: partition values.
percent_change_numbers	Standard-library collection processing operation: percent change numbers.
product_numbers	Standard-library collection processing operation: product numbers.
quantile_number	Standard-library collection processing operation: quantile number.
reject_values	Standard-library collection processing operation: reject values.
reverse	Return the items of a list in reverse order.
reverse_range_values	Standard-library collection processing operation: reverse range values.
reverse_values	Standard-library collection processing operation: reverse values.
rotate_values	Standard-library collection processing operation: rotate values.
running_values	Standard-library collection processing operation: running values.
running_values_right	Standard-library collection processing operation: running values right.
scan_values	Standard-library collection processing operation: scan values.
scan_values_right	Standard-library collection processing operation: scan values right.
skip	Return the list after skipping the requested number of items.
sliding_windows	Standard-library collection processing operation: sliding windows.
sort_by_values	Standard-library collection processing operation: sort by values.
sort_descending_values	Standard-library collection processing operation: sort descending values.
sort_numbers	Return numbers in ascending order.
sort_values	Standard-library collection processing operation: sort values.
span_while	Standard-library collection processing operation: span while.

Function	Description
split_at_values	Standard-library collection processing operation: split at values.
stddev_number	Standard-library collection processing operation: stddev number.
sum_numbers	Return the sum of a list of numbers.
sum_squares_numbers	Standard-library collection processing operation: sum squares numbers.
symmetric_difference_values	Standard-library collection processing operation: symmetric difference values.
take	Return the first requested number of list items.
take_every_from_values	Standard-library collection processing operation: take every from values.
take_every_values	Standard-library collection processing operation: take every values.
take_until	Standard-library collection processing operation: take until.
take_while	Standard-library collection processing operation: take while.
union_values	Standard-library collection processing operation: union values.
unique_values	Standard-library collection processing operation: unique values.
unzip_values	Standard-library collection processing operation: unzip values.
variance_number	Standard-library collection processing operation: variance number.
weighted_average_numbers	Standard-library collection processing operation: weighted average numbers.
z_scores_numbers	Standard-library collection processing operation: z scores numbers.
zip_values	Standard-library collection processing operation: zip values.

13.2.4 csv — CSV input and output

Function	Description
read_csv	read UTF-8 CSV into records
write_csv	write records as UTF-8 CSV

13.2.5 data — Record and field access

Function	Description
get	Return the value associated with a record field.
has	Check whether a record contains a field.
keys	Return the field names in a record.
merge_records	Combine the fields of two records.

Function	Description
pluck	Extract one field from each record in a list.
remove_field	Standard-library record and field access operation: remove field.
select_fields	Standard-library record and field access operation: select fields.

13.2.6 datetime — Date and time operations

Function	Description
add_days_datetime	Standard-library date and time operation: add days datetime.
add_months_datetime	Standard-library date and time operation: add months datetime.
add_seconds_datetime	Standard-library date and time operation: add seconds datetime.
date_components_datetime	Standard-library date and time operation: date components datetime.
days_in_month	Standard-library date and time operation: days in month.
diff_seconds_datetime	Standard-library date and time operation: diff seconds datetime.
end_of_day_datetime	Standard-library date and time operation: end of day datetime.
format_datetime	Standard-library date and time operation: format datetime.
is_leap_year	Standard-library date and time operation: is leap year.
now_utc	read the current UTC instant as ISO-8601 text
parse_datetime	Standard-library date and time operation: parse datetime.
start_of_day_datetime	Standard-library date and time operation: start of day datetime.
today_utc	read the current UTC date as YYYY-MM-DD
week_number_datetime	Standard-library date and time operation: week number datetime.
weekday_datetime	Standard-library date and time operation: weekday datetime.

13.2.7 files — File and directory operations

Function	Description
append_text	append UTF-8 text to a file
copy_file	copy a file
exists	test whether a path exists
file_size	read file size
is_directory	test whether a path is a directory

Function	Description
is_file	test whether a path is a file
list_directories	list directories directly contained in a directory
list_directory	list directory entries
list_files	list files directly contained in a directory
make_directories	create a directory tree
make_directory	create a directory
move_file	move a file
read	read UTF-8 text from a file
read_lines	read a UTF-8 text file as lines
read_text	read UTF-8 text from a file
remove_directory	remove an empty directory
remove_directory_tree	remove a directory tree
remove_file	remove a file
write	write UTF-8 text to a file
write_lines	write text lines to a file
write_text	write UTF-8 text to a file

13.2.8 json — JSON parsing and serialization

Function	Description
json_get	Standard-library JSON data operation: json get.
json_has	Standard-library JSON data operation: json has.
json_parse	Parse JSON text into an S-Lang value.
json_pretty	Convert a value to indented, readable JSON text.
json_set	Standard-library JSON data operation: json set.
json_stringify	Convert a value to compact JSON text.
read_json	read JSON from a UTF-8 file
write_json	write JSON to a UTF-8 file

13.2.9 math — Mathematical functions

Function	Description
abs_number	Standard-library mathematics operation: abs number.
acos_number	Standard-library mathematics operation: acos number.
acosh_number	Standard-library mathematics operation: acosh number.
asin_number	Standard-library mathematics operation: asin number.
asinh_number	Standard-library mathematics operation: asinh number.
atan2_number	Standard-library mathematics operation: atan2 number.
atan_number	Standard-library mathematics operation: atan number.

Function	Description
atanh_number	Standard-library mathematics operation: atanh number.
cbrt_number	Standard-library mathematics operation: cbrt number.
ceil	Standard-library mathematics operation: ceil.
clamp_number	Standard-library mathematics operation: clamp number.
cos_number	Standard-library mathematics operation: cos number.
cosh_number	Standard-library mathematics operation: cosh number.
degrees_number	Standard-library mathematics operation: degrees number.
exp2_number	Standard-library mathematics operation: exp2 number.
exp_number	Standard-library mathematics operation: exp number.
factorial_number	Standard-library mathematics operation: factorial number.
floor	Standard-library mathematics operation: floor.
fraction_number	Standard-library mathematics operation: fraction number.
gcd_number	Standard-library mathematics operation: gcd number.
hypot_number	Standard-library mathematics operation: hypot number.
is_even_number	Standard-library mathematics operation: is even number.
is_finite_number	Standard-library mathematics operation: is finite number.
is_integer_number	Standard-library mathematics operation: is integer number.
is_nan_number	Standard-library mathematics operation: is nan number.
is_odd_number	Standard-library mathematics operation: is odd number.
lcm_number	Standard-library mathematics operation: lcm number.
lerp_number	Standard-library mathematics operation: lerp number.
log10_number	Standard-library mathematics operation: log10 number.
log2_number	Standard-library mathematics operation: log2 number.
log_number	Standard-library mathematics operation: log number.
max_of_numbers	Standard-library mathematics operation: max of numbers.
min_of_numbers	Standard-library mathematics operation: min of numbers.

Function	Description
mod_number	Standard-library mathematics operation: mod number.
normalize_number	Standard-library mathematics operation: normalize number.
pow_number	Standard-library mathematics operation: pow number.
radians_number	Standard-library mathematics operation: radians number.
reciprocal_number	Standard-library mathematics operation: reciprocal number.
remap_number	Standard-library mathematics operation: remap number.
round_number	Standard-library mathematics operation: round number.
sign_number	Standard-library mathematics operation: sign number.
sin_number	Standard-library mathematics operation: sin number.
sinh_number	Standard-library mathematics operation: sinh number.
sqrt	Return the square root of a number.
tan_number	Standard-library mathematics operation: tan number.
tanh_number	Standard-library mathematics operation: tanh number.
truncate_number	Standard-library mathematics operation: truncate number.

13.2.10 path — File path handling

Function	Description
basename	Standard-library file path handling operation: basename.
dirname	Standard-library file path handling operation: dirname.
extension	Standard-library file path handling operation: extension.
join_path	Standard-library file path handling operation: join path.
normalize_path	Standard-library file path handling operation: normalize path.

13.2.11 process — Process and environment management

Function	Description
argument_at	Standard-library process and environment management operation: argument at.
argument_count	Standard-library process and environment management operation: argument count.
arguments	Standard-library process and environment management operation: arguments.
command_exists	Standard-library process and environment management operation: command exists.
current_directory	Standard-library process and environment management operation: current directory.
environment	Standard-library process and environment management operation: environment.
environment_or	Standard-library process and environment management operation: environment or.
executable_path	Standard-library process and environment management operation: executable path.
has_environment	Standard-library process and environment management operation: has environment.
home_directory	Standard-library process and environment management operation: home directory.
platform_name	Standard-library process and environment management operation: platform name.
process_id	Standard-library process and environment management operation: process id.
remove_environment	Standard-library process and environment management operation: remove environment.
run_process	Standard-library process and environment management operation: run process.
run_process_timeout	Standard-library process and environment management operation: run process timeout.
run_program	Run a program with an argument list.
run_program_timeout	Run a program with a time limit.
run_program_with_environment	Standard-library process and environment management operation: run program with environment.
set_environment	Standard-library process and environment management operation: set environment.
sleep_seconds	Standard-library process and environment management operation: sleep seconds.
temp_directory	Standard-library process and environment management operation: temp directory.

13.2.12 relational — Relational data operations

Function	Description
count_by	Standard-library relational data operations operation: count by.
distinct_by	Keep one record for each distinct field value.
find_by	Standard-library relational data operations operation: find by.

Function	Description
join_records	Join two record lists using matching key fields.
order_by	Sort records by a selected field.
partition	Standard-library relational data operations operation: partition.

13.2.13 text — Text processing

Function	Description
char_at_text	Standard-library text processing operation: char at text.
contains_text	Standard-library text processing operation: contains text.
count_text	Standard-library text processing operation: count text.
ends_with	Standard-library text processing operation: ends with.
index_of	Standard-library text processing operation: index of.
join_lines	Standard-library text processing operation: join lines.
join_texts	Join a list of text values with a separator.
length_text	Standard-library text processing operation: length text.
lower	Convert text to lowercase.
normalize_whitespace	Standard-library text processing operation: normalize whitespace.
pad_left_text	Standard-library text processing operation: pad left text.
pad_right_text	Standard-library text processing operation: pad right text.
remove_prefix_text	Standard-library text processing operation: remove prefix text.
remove_suffix_text	Standard-library text processing operation: remove suffix text.
repeat_text	Standard-library text processing operation: repeat text.
replace	Standard-library text processing operation: replace.
reverse_text	Standard-library text processing operation: reverse text.
split	Split text into a list using a separator.
split_lines	Standard-library text processing operation: split lines.
starts_with	Standard-library text processing operation: starts with.
substring_text	Standard-library text processing operation: substring text.
trim	Remove whitespace from the beginning and end of text.

Function	Description
upper	Convert text to uppercase.

Chapter 14

Command-Line Reference

Command	Purpose
`slang version`	Print the public compiler version.
`slang check FILE`	Parse and semantically validate a source file.
`slang run FILE`	Run a source file with the interpreter.
`slang run --native-jit FILE`	Run a supported program through the in-process LLVM Native JIT.
`slang backend-capabilities FILE`	Report backend capabilities for a source program.
`slang init DIR`	Create a new S-Lang project.
`slang lsp`	Start the Language Server Protocol service over standard input/output.
`slang stdlib`	Inspect the public standard-library surface.

Chapter 15

Diagnostics and Validation Workflow

Use the following order when a program does not work:

1. Run `slang version` and confirm that the expected release is installed.
2. Run `slang check FILE`. Fix syntax, name-resolution, and type errors before testing runtime behavior.
3. Run `slang run FILE` to check interpreter behavior.
4. If using Native JIT, inspect `slang backend-capabilities FILE` and then run `slang run --native-jit FILE`.
5. When reporting a defect, include a minimal source file, the exact command, the complete diagnostic, and the operating system.

A successful syntax check does not prove that every backend can execute a construct. Backend support is a separate contract.

Chapter 16

Forms Not Included in S-Lang 1.0

The following forms are not part of the language grammar described by this release:

- `for` and `while` statement loops;
- lambda expressions such as `x -> x * 2`;
- list indexing such as `values[0]`;
- multi-line pipelines split across physical lines;
- `fn` as an alternative function declaration keyword;
- bare assignment as a universal variable declaration;
- a top-level `return` outside a valid function body.

Do not copy syntax from another language unless it is explicitly documented for S-Lang.

Chapter 17

Complete Example

```
data students:
    Alice age 22 score 91
    Bob age 17 score 88
    Carol age 20 score 95
    David age 19 score 84

let adults = students where age >= 18
let ranked = adults sort by score desc
let scores = ranked map score
let total = scores sum item
let average_score = scores average item
show ranked
show total, average_score
```

This example demonstrates data declarations, record-field shorthand in collection context, filtering, sorting, mapping, aggregation, and output.

Chapter 18

Practice Exercises

18.1 Beginner

1. Declare two numbers and display their sum and product.
2. Use a conditional expression to classify a score as `pass` or `fail`.
3. Declare a mutable counter, update it twice, and display its final value.

18.2 Intermediate

1. Declare a typed function that converts Celsius to Fahrenheit.
2. Define a `Person` record and create two instances.
3. Filter a numeric list, double the selected values, sort them, and calculate their sum.

18.3 Applied

1. Read a CSV file, filter its records, and calculate an aggregate.
2. Create a module that exports one public function and import it from another source file.
3. Compare interpreter and Native JIT behavior for a program supported by both backends.

Chapter 19

Language Reference and Change Control

The parser in `slang/core.py` defines accepted syntax. Semantic and type rules are implemented in `slang/semantic.py`, `slang/types.py`, and `slang/ast.py`. Runtime behavior is defined by the interpreter, native runtime, standard-library implementation, and tests. Editor grammars provide highlighting and language tooling; they are not normative specifications.

A language change is complete only when the implementation, positive and negative conformance tests, runtime tests, editor integrations, and documentation agree. If a construct is not listed as supported, do not assume it is valid.

Release Information

S-Lang 1.0 is the stable public version line. The source archive includes the compiler, runtime, standard library, tests, examples, editor integrations, packaging scripts, and build workflows. The Windows runtime executable is named `slang1.0.exe`. End users installing a prebuilt package do not need to install Python, GCC, Clang, or LLVM separately to run the language.